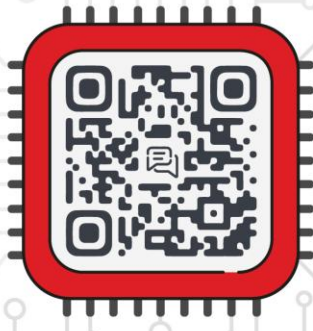




نظري

22

8

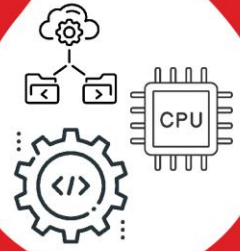
3300 sp

كلية الهندسة المعلوماتية

السنة الرابعة

استراتيجيات إدارة التنافس  
وتحسين الأداء

د. عمار جوخدار



برمجة تفرعية

محتوى مجاني غير مخصص للبيع التجاري

**RB** Informatics; 18/5/2026

في الأنظمة المؤسسية والتطبيقات الشبكية متعددة المستخدمين، تتشارك أعداد كبيرة من المستخدمين والعمليات في الوصول إلى نفس المَوارِد والبيانات في الوقت نفسه. وقد يؤدي هذا التشارك إلى حدوث حالات تنافس تؤثر في صحة البيانات وأداء النظام إذا لم تتم إدارتها بشكل صحيح. لذلك تعتمد هذه الأنظمة على مجموعة من الآليات والتقنيات التي تضمن تنظيم الوصول إلى المَوارِد المشتركة، والمحافظة على سلامة البيانات، وتحسين استهلاك المَوارِد والأداء، لتتغرف عليها في هذه المحاضرة.

▪ لنبدأ محاضرتنا بالإجابة على سؤال:

❧ من أين ظهرت مشكلة التنافس؟

تَظهر مُشكلة التنافس

عندما تعمل عدّة Threads بشكل مُتزامن أو متوازٍ و تحتاج جميعها إلى الوصول إلى المَوارِد أو البيانات نفسها لتنفيذ المهام المطلوبة منها. وعند محاولة أكثر من Thread استخدام المورد **نفسه في الوقت ذاته**، قد يحدث تضارب يؤثر في صحة البيانات وأداء النظام ما لم يتم استخدام آليات مناسبة لتنظيم هذا الوصول.

❖ تُعدّ مشكلة التنافس من المشكلات الشائعة في تطبيقات الويب، وذلك لأن تطبيقات الويب الحديثة مُصمّمة **للمعالجة عدّة طلبات (Requests)** بشكل مُتزامن. فعند وصول عدد من الطلبات من المستخدمين في الوقت نفسه، يتولى خادم الويب (Web Server) والبنية التشغيلية للتطبيق إدارة هذه الطلبات و**مُعالجتها بالتوازي** أو بشكل مُتزامن، **دون أن يضطر المُبرمج** إلى إنشاء Threads **يُحوّل** للتعامل مع كُل مُستخدم. لذلك فإن مُعظم الأنظمة التي تعتمد على خادم ويب تعمل بطبيعتها في بيئة متعددة المهام، مما يجعل عدّة Requests قادرة على الوصول إلى البيانات أو المَوارِد نفسها في الوقت ذاته، وبالتالي تصبح مُشكلة التنافس (Concurrency) **جزءاً طبيعياً** من عمل تطبيقات الويب ويجب أخذها بالحسبان عند تصميم النظام.



إن إنشاء وإدارة الـ Threads لا يُعدّ قراراً برمجياً بسيطاً

بل يُعتبر **قراراً تصميمياً** يؤثر بشكل مباشر في أداء النظام واستهلاكه للموارد. فعملية إنشاء Thread جديد وإتلافه عملية مُكلفة نسبياً من حيثُ الزمن والذاكرة، كما أن **لكل Thread دورة حياة** تحتاج إلى إدارة ومُراقبة. لذلك **لا يُنصح** عادةً بأن يقوم المُبرمج بإنشاء Threads بشكل مباشر لكل مُهمّة، بل تعتمد الأنظمة المؤسسية على حاويات (Containers) أو بيئات تشغيل متخصصة **تتولى إدارة الـ Threads** والتحكم بدورة حياتها.

وتُعدّ Thread Pools إحدى الآليات المُستخدمة لتحقيق ذلك

حيثُ يتم إنشاء مجموعة من الـ Threads  **وإعادة استخدامها** بدلاً من إنشاء Thread جديد لكل طلب. إلا أن إدارة التزامن في الأنظمة المؤسسية لا تقتصر على Thread Pools فقط، بل تشمل أيضاً **مُراقبة الـ Threads**، وإعادة تشغيل الخدمات المُتوقّفة، **وزيادة أو تخفيض** عدد الـ Workers أو Threads وفقاً لحجم الحمل على النظام.

ولهذا نجد أن خوادم الويب (Web Servers)

قادرة على التعامل مع أعداد كبيرة **من الطلبات بشكل مستمر**، لأنها تعتمد على آليات **إدارة مُتقدّمة** للـ Threads والعمليات، بينما **قد تتوقف** بعض التطبيقات متعددة الخيوط التي يُطوّرُها المُبرمجون مباشرةً **إذا لم تتوفّر** جهة **مسؤولة عن مُراقبة الخيوط** وإدارة دورة حياتها واستعادتها عند حدوث الأخطاء.

## لماذا نهتم بالتوازي (Parallel Processing)؟

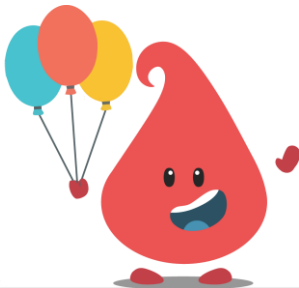
لأن معظم الأنظمة الحديثة التي نتعامل معها اليوم تعمل بشكل مُتزامن ومُتوازي لِخدمة عدد كبير من المُستخدمين في الوقت نفسه. لكن تشغيل عدّة Threads بشكل مُتواز ليس كافياً بحد ذاته، **بل يجب ضمان** إدارة هذه الـ Threads وعدم تركها تتوقف أو تبقى معلقة بشكل دائم.

لذلك لا يقوم المُبرمج عادةً بإنشاء وإدارة الـ Threads بنفسه

وإنما **يُوكّل هذه المُهمّة** إلى الويب سيرفر أو الحاوية (Container) التي تُدير دورة حياة الـ Threads، وتتحكم بعددها وتُراقب حالتها.

فعلى سبيل المثال: قد يكون الحد الأدنى لعدد الـ Threads هو 20 والحد الأقصى 100. وعند وصول عدد كبير من المُستخدمين **بشكل مُتزامن**، تتم معالجة عدد من الطلبات مباشرة، بينما تنتظر الطلبات الزائدة ضمن طابور انتظار (Queue). كما **لا يُسمح** لهذه الطلبات بالانتظار إلى أجل غير مُسمّى...

بل **يتم تحديد مهلة انتظار مُعيّنة**، فإذا انتهت هذه المهلة دون توفّر Thread لمعالجة الطلب، **يتم رفض الطلب** بدلاً من إبقائه مُنتظراً إلى ما لا نهاية، وذلك حفاظاً على استقرار النظام وموارده.



**ملاحظة اربيسرية:**

من المهم التمييز بين إدارة الـ Threads وإدارة التنافس، حيث تتولى أطر العمل Frameworks وخوادم الويب إدارة الخيوط ودورة حياتها، أما مسؤولية معالجة التنافس على البيانات المشتركة فهي من مسؤوليات المبرمج.

**من الأمثلة الشائعة على مشكلة التنافس :**

■ في الأنظمة المصرفية، إذا حاول شخصان إجراء عملية سحب من الحساب نفسه في اللحظة ذاتها، فإنهما سيتنافسان على الوصول إلى الرصيد وتعديله، مما قد يؤدي إلى نتائج خاطئة إذا لم يتم تنظيم هذا الوصول بشكل صحيح.

■ ومن الأمثلة الأخرى مُحركات الألعاب (Game Engines)، حيث يعمل نظام التحريك (Physics أو Movement Engine) ونظام الاظهار (Rendering Engine) بشكل متزامن. فبينما يقوم نظام

التحريك بتحديث موقع

السيارة من النقطة A

إلى النقطة C، قد يحاول

نظام الاظهار قراءة

موقع السيارة ورسمها

على الشاشة في اللحظة

نفسها. وإذا تمت

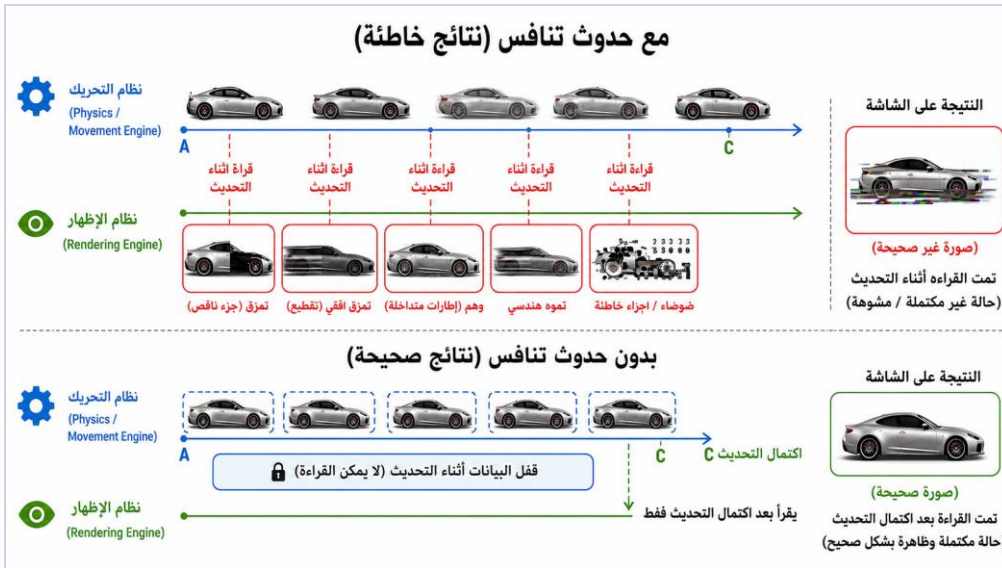
القراءة أثناء عملية

التحديث، فقد تظهر

السيارة بشكل مشوه أو

في حالة غير مكتملة،

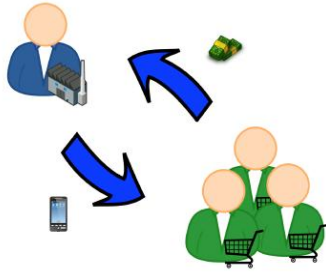
لأن أحد النظامين يقرأ البيانات بينما يقوم الآخر بتعديلها.

**معالجة التنافس Concurrent Access**

لمعالجة مشكلة التنافس تُستخدم مجموعة من الآليات التي تُنظّم الوصول إلى البيانات المَورِدَة المشتركة وتمنع حدوث التضارب بينها. ويُعدّ التزامن (Synchronization) أحد أشهر هذه الحُلُول، ومن نماذج التصميم الشائعة (Design Patterns) المُستخدمة في مُعالجة هذه المشكلة :

**1. نموذج المُنتِج والمُسْتَهِك (Producer/Consumer) :**

يُعدّ هذا النموذج من أشهر النماذج المُستخدمة لمُعالجة مشكلة التنافس في الأنظمة البرمجية. تقوم فكرته على وجود طرف مُنتِج (Producer) يقوم بإنتاج البيانات أو الرسائل وطرف مُستهلك يقوم بقراءتها أو مُعالجتها.



Producer - Consumer

ويهدف هذا النموذج إلى تنظيم عملية تبادل البيانات بين الطرفين ومنع حدوث التضارب أثناء العمل المتوازي.

#### ■ أمثلة على النموذج:

■ يُمكن تطبيق هذا النموذج على مثال مُحركات الألعاب، إذ يُعدّ نظام التحريك مُنتجاً للبيانات (Producer)، في حين يُعدّ نظام الاظهار مُستهلكاً لها (Consumer).

■ أيضاً من الأمثلة الشائعة على نموذج المُنتج والمُستهلك أنظمة البيع الإلكتروني. ففي هذه الأنظمة تُعدّ عملية الشراء والدفع الإلكتروني بمثابة المُنتج (Producer)، بينما تُعدّ أنظمة التنفيذ والشحن بمثابة المُستهلك (Consumer). لذلك لا يجوز أن تبدأ عملية التنفيذ أو تسليم البضاعة قبل اكتمال عملية الشراء بشكل كامل.

■ فعلى سبيل المثال، إذا قام أحد المُستخدمين بإجراء عملية شراء عبر الإنترنت، فإنّ المُستودع لا يجب أن يبدأ بتجهيز الطلب إلا بعد نجاح عملية الدفع وتثبيت عملية البيع في قاعدة البيانات (Commit). أما إذا تمكّن نظام التنفيذ من رؤية البيانات قبل اكتمال المُعاملة (Transaction)، بسبب استخدام مُستوى عزل (Isolation Level) يسمح بقراءة البيانات غير المؤكدة (Dirty Read)، فقد يعتقد أنّ عملية البيع تمت بالفعل ويبدأ بشحن البضاعة رغم أن عملية الدفع فشلت لاحقاً. وعندها تكون البضاعة قد سلّمت دون عملية شراء حقيقية.

■ ولهذا السبب تُستخدم المُعاملات (Transactions) ومُستويات العزل (Isolation Levels) المناسبة لضمان عدم انتقال البيانات من المُنتج إلى المُستهلك إلا بعد اكتمال العملية بشكل صحيح، مما يمنع حدوث مثل هذه الأخطاء ويحافظ على سلامة النظام.

#### ■ ولكي يعمل هذا النموذج بشكل صحيح لا بُد من تحقيق ثلاثة شروط أساسية :

1. أولاً: لا يجوز للمُستهلك استهلاك مُنتج غير مُكتمل، أي يجب أن ينتهي المُنتج (Producer) من إنتاج البيانات بشكل كامل قبل أن تُصبح متاحة للاستهلاك.
2. ثانياً: لا يجوز للمُنتج (Producer) إنتاج عنصر جديد قبل استهلاك العنصر السابق كاملاً من قبل المُستهلك ، وذلك لمنع ضياع البيانات أو الكتابة فوق بيانات لم تتم معالجتها بعد.
3. ثالثاً: لا يجوز للمُستهلك استهلاك المُنتج نفسه أكثر من مرة، لأنّ ذلك يؤدي إلى تكرار تنفيذ العملية نفسها.

- وتظهر **أهمية هذه الشروط** بوضوح في الأنظمة المؤسسية وأنظمة التجارة الإلكترونية، حيث تنتقل البيانات داخل النظام على شكل رسائل (Messages) بين المكونات والخدمات المختلفة.
- فعلى سبيل المثال: إذا تم استهلاك رسالة دفع إلكتروني مرتين فقد يؤدي ذلك إلى تنفيذ عملية السحب أو الدفع مرتين، وهو خطأ خطير. كما لا يجوز تجاهل الرسالة وعدم استهلاكها نهائياً، لأن ذلك يعني ضياع العملية المطلوبة. لذلك يجب ضمان أن كل رسالة :  
 يتم إنتاجها مرة واحدة فقط ، وتستهلك مرة واحدة فقط ، وذلك بعد اكتمال إنتاجها بالكامل ،  
 وعندها فقط يمكن للنظام أن يعمل بشكل صحيح وآمن دون حدوث مشاكل تنافس أو فقدان للبيانات.

- في الأنظمة المؤسسية **يقع على عاتق المهندس** مسؤولية تصميم البنية العامة للنظام (Architecture) بحيث يضمن انتقال البيانات والرسائل بين المكونات المختلفة **بطريقة صحيحة وآمنة**. ويجب أن يضمن التصميم تنفيذ كل رسالة مرة واحدة فقط **وعدم ضياعها** أو تكرار تنفيذها، لأن تكرار الرسائل قد يؤدي إلى تنفيذ العملية نفسها أكثر من مرة، بينما يؤدي فقدانها إلى ضياع جزء من العمل المطلوب. وعند حدوث أي عطل أو سلوك غير متوقع، يجب أن يكون التصميم واضحاً بما يكفي **لتحديد مكان الخلل** ومعالجته دون التأثير على سلامة البيانات أو استقرار النظام.

## مثال مسألة المنتج/المستهلك

```

1 class Product {
2     int p[10];
3     int k = 0;
4     void produce() {
5         for (int i = 0; i < 10; i++) {
6             p[i] = k;
7         }
8         k++;
9     }
10    void consume() {
11        System.out.println(p);
12    }
13 }

```

```

1 public static void main(String[] args) {
2     Product p = new Product();
3     Producer pr = new Producer(p);
4     Consumer c = new Consumer(p);
5     pr.start();
6     c.start();
7 }

```

- في هذا المثال لدينا كائن مشترك من الصنف Product يمثل المورد المشترك الذي لا نريد أن يحدث عليه تنافس، و يحتوي على مصفوفة من العناصر p[10]، بالإضافة إلى دالتين هما produce() و consume().
- تقوم الدالة produce() بتعديل **بيانات المصفوفة** وزيادة قيمها.
- بينما تقوم الدالة consume() بقراءة هذه البيانات وعرضها.
- لو تم تنفيذ العمليتين بشكل متسلسل (Sequential)، فإن المنتج سيُنتج البيانات أولاً ثم يأتي المستهلك ويقرأها، وبالتالي سنحصل على نتائج مرتبة ومتوقعة (000111222..).

لكن هذه الحالة لا تُحقق الاستفادة الحقيقية من قدرات الحاسب، لأن المُنتِج والمُستهلك سيكونان عملياً جزءاً من خيط تنفيذ واحد (Single Thread أو كتسمية أخرى Mono Thread)، ولن نستفيد من تعدد الأنوية أو تعدد الخوادم أو أيّاً كانت { المُتاحة.

## فصل الشريدات:

```
1 class Consumer extends Thread {
2     Product p;
3
4     Consumer(Product p) {
5         this.p = p;
6     }
7
8     public void run() {
9         while (true) {
10            p.consume();
11        }
12    }
13 }
```

```
1 class Producer extends Thread {
2     Product p;
3
4     Producer(Product p) {
5         this.p = p;
6     }
7
8     public void run() {
9         while (true) {
10            p.produce();
11        }
12    }
13 }
```

لذلك يتم تشغيل المُنتِج والمُستهلك ضمن خيوط تنفيذ مُستقلة (Multi Threads). ففي الدالة main يتم إنشاء كائن واحد من الصنف Product وهو p، ثم تمرير المرجع (instance) نفسه إلى كل من Producer و Consumer.

بعد ذلك يتم تشغيلهما باستخدام start() فيصبح لدينا Thread خاص بالمنتج و Thread خاص بالمستهلك، ويعمل كل منهما بشكل مُستقل وعلى التوازي مع الآخر، مع تشاركهما في الوصول إلى المورد نفسه وهو الكائن Product.

تُكمن المشكلة في أن المُنتِج والمُستهلك لا يُنسّقان عملهما مع بعضهما البعض. فالمُنتِج قد يكون ما زال يُعدّل البيانات ويكتب عليها، بينما يحاول المُستهلك قراءتها في اللحظة نفسها. وبما أن التنفيذ أصبح متوازياً، فإن المُستهلك قد يقرأ البيانات قبل اكتمال عملية الإنتاج أو أثناء تعديلها، فتظهر نتائج غير صحيحة أو غير مُتناسقة. فبدل أن نرى القيم تنتقل بشكل منطقي من حالة إلى أخرى، قد نحصل على مُخرجات عشوائية تحتوي على أجزاء من بيانات قديمة وأخرى جديدة (0002222114433335)، لأن عملية القراءة تمت أثناء عملية الكتابة. وهنا تظهر مشكلة التنافس (Race Condition).



## حل مشكلة التنافس في نموذج المُنتج/المُستهلك :

■ وذلك بإضافة synchronized :

■ لحل مشكلة التنافس في المثال السابق نبدأ من الصنف Product، لأنه يُمثّل المورد المُشترك بين المُنتج والمُستهلك. و أول حل يُمكن استخدامه هو إضافة الكلمة synchronized قبل الدالتين produce() و consume()، وبذلك نحقق نوعاً من التزامن (Synchronization) بين الخيوط.

■ synchronized تعني أن الخيط الذي يدخل إلى دالة مُتزامنة داخل الكائن يحصل على قفل هذا الكائن (الذي تنتمي له هذه الدالة) وبالتالي لا يستطيع أي خيط آخر الدخول إلى أي دالة أخرى مُتزامنة على الكائن نفسه حتى ينتهي الخيط الأول ويحرر القفل. وبهذا نمنع المُنتج والمُستهلك من الوصول إلى بيانات Product في اللحظة نفسها بطريقة غير آمنة.

■ لكن هذا الحل ليس مثالياً دائماً، لأنه يجعل الخيوط تنتظر بعضها وتعمل بالتتالي داخل الأجزاء المُتزامنة. فإذا جعلنا كل العمل داخل النظام متزامناً، فإننا نعود

عملياً إلى حالة قريبة من التنفيذ أحادي الخيط (Single Thread)، ونفقد فائدة العمل المتوازي. لذلك يُمنع منعاً باتاً استخدام التزامن من قبل المُبرمج إلا بأمر من المُصمّم، أي يجب أن يكون ذلك بناءً على قرار تصميمي واضح يُحدد أين توجد مناطق التنافس بالضبط.

■ والأفضل هو جعل الجزء الصغير الذي يتعامل مع المورد المشترك فقط متزامناً (Critical Section)، بينما تبقى بقية أجزاء العمل غير مُتزامنة لتستفيد من التوازي. فمثلاً عند العمل بطريقة غير مُتزامنة (Asynchronous)، غالباً ما يوجد طابور (Queue) توضع فيه العناصر التي يُنتجها المُنتج بانتظار أن يَستهلكها المُستهلك. في هذه الحالة يكون الطابور هو المورد المشترك والمنطقة الحساسة التي قد يحدث عليها التنافس، لذلك نُنسق الوصول إليه فقط ونترك بقية العمليات تعمل بشكل متوازٍ. أما إذا كان أحد الطرفين أسرع من الآخر، فهذه ليست مشكلة بحد ذاتها ما دامت آلية التنسيق تضمن عدم استهلاك بيانات غير مُكتملة وعدم ضياع البيانات أو تكرار استهلاكها.

■ في هذا المثال استطاعت synchronized حل مشكلة استهلاك مُنتج غير مُكتمل. فعندما يدخل الـ Producer إلى الدالة produce() ويحصل على القفل (Lock)، لا يستطيع الـ Consumer الدخول إلى الدالة consume() طالما أن الـ Producer ما زال يعمل داخل القسم المتزامن (Critical Section). لذلك يضطر الـ Consumer إلى الانتظار حتى ينتهي الـ Producer من عمله ويحرر القفل، وعندها فقط يُسمح له بالدخول وقراءة البيانات. وبالمثل، إذا دخل الـ Consumer أولاً إلى الدالة consume()، فلن يتمكن الـ Producer من

```

1 class Product {
2     int p[10];
3     int k = 0;
4     synchronized void produce() {
5         for (int i = 0; i < 10; i++) {
6             p[i] = k;
7         }
8         k++;
9     }
10    synchronized void consume() {
11        System.out.println(p);
12    }
13 }

```

الدخول إلى الدالة `produce()` حتى ينتهي الـ `Consumer` ويحرر القفل. وبهذا تضمن آلية التزامن عدم وجود عمليتي قراءة وكتابة على المورد المشترك في اللحظة نفسها.

- تتولى بيئة التنفيذ الخاصة بلغة البرمجة (Runtime Environment) إدارة هذه العملية، حيث تقوم بتنظيم دخول الـ `Threads` إلى الأقسام المتزامنة ومنع القفل لخطط تنفيذ واحد فقط في كل مرة. لذلك فإن استخدام `synchronized` لا يمنع وجود عدة `Threads`، وإنما يمنع دخولها المتزامن إلى الجزء الحساس من البرنامج، فتعمل هذه الأجزاء بشكل متتالي بينما تبقى بقية أجزاء التطبيق قادرة على العمل بالتوازي.
- عند الالتزام بنموذج المنتج والمستهلك، يجب أن تكون حماية المورد المشترك موجودة داخل الصنف الذي يمثل هذا المورد، أي داخل `Product` نفسه، وليس داخل خيوط التنفيذ `Producer` و `Consumer` فالمنتج والمستهلك يستخدمان المورد المشترك، أما مسؤولية تنظيم الوصول إليه فيجب أن تكون ضمن المورد نفسه. لذلك أضفنا `synchronized` إلى الدوال التي يحدث فيها تداخل، مما منع الـ `Threads` من الدخول إلى هذه الدوال في الوقت نفسه.
- لكن رغم أن `synchronized` يمنع القراءة والكتابة المتزامنة على المورد، إلا أنها لا تكفي وحدها لضمان صحة التسلسل المنطقي بين الإنتاج والاستهلاك.
  - فقد يكون المنتج أسرع من المستهلك، فينتج أكثر من مرة قبل أن يستهلك المستهلك المنتج السابق.
  - أو قد يحاول المستهلك الاستهلاك أكثر من مرة.
 لذلك نضيف متغير حالة مثل `isAvailable` داخل الصنف `Product`.

معالجة استهلاك منتج مرتين أو استهلاك منتج غير كامل باستعمال الانتظار الفعّال ( `Busy Waiting`):

```

1 class Product {
2     int p[10];
3     int k = 0;
4     boolean isAvailable=false;
5
6     synchronized void produce() {
7         if (isAvailable)
8             { return; }
9         for (int i = 0; i < 10; i++)
10            { p[i]=k; }
11            k++;
12        }
13
14        isAvailable=true;
15    }

```

```

1 synchronized void consume() {
2     if (!isAvailable)
3         { return; }
4     System.out.println(p);
5     isAvailable=false;
6 }

```

- تكون قيمة `isAvailable` في البداية `false`، أي لا يوجد منتج جاهز للاستهلاك. وعندما تنتهي دالة `produce()` من إنتاج المنتج تصبح القيمة `true`، وعندما تُنفَّذ دالة `consume()` عملية الاستهلاك تعود القيمة إلى `false`. وبذلك يعرف كل طرف متى يُسمح له بالتنفيذ ومتى يجب عليه الانتظار.

- **إلا أن هذا الحل يعتمد على ما يُسمى بالانتظار الفعّال (Busy Waiting)**، حيث يستمر المُستهلك في فحص قيمة `isAvailable` بشكل متكرر لمعرفة ما إذا أصبح المُنتج جاهزاً أم لا. ورغم أن هذا الأسلوب يعمل بشكل صحيح، إلا أنه يَستهلك وقت المُعالج دون فائدة حقيقية. لذلك يُستخدم أسلوب أكثر كفاءة يعتمد على الدالة `wait()`.
- فعندما يجد المُستهلك أن المُنتج غير جاهز، **لا يستمر في الدوران** ضمن حلقة فحص مستمرة، وإنما يستدعي `wait()` فيُزال من المُعالج وينتقل إلى حالة انتظار (Waiting State). ويبقى خارج التنفيذ إلى أن ينتهي المُنتج من عمله ويستدعي `notify()` أو `notifyAll()`، **وعندها يتم إشعار المُستهلك** بأن المُنتج أصبح جاهزاً، فيعود إلى التنفيذ ويكمل عمله. وبهذه الطريقة نتجنب إستهلاك موارد المُعالج أثناء فترات الانتظار ونُحافظ على كفاءة النظام.
- **مُعالجة استهلاك مُنتج مرتين أو عدم استهلاكه باستعمال حالات الترقب (Monitoring Status):**

```

1 class Product {
2     int p[10];
3     int k = 0;
4     boolean isAvailable = false;
5
6     synchronized void produce() {
7         while (isAvailable) {
8             wait();
9         }
10        for (int i = 0; i < 10; i++) {
11            p[i] = k;
12        }
13        k++;
14    }
15    isAvailable=true;
16    notifyAll();
17 }

```

```

1 synchronized void consume() {
2     while (!isAvailable) {
3         wait();
4     }
5     System.out.println(p);
6     isAvailable = false;
7     notifyAll();
8 }

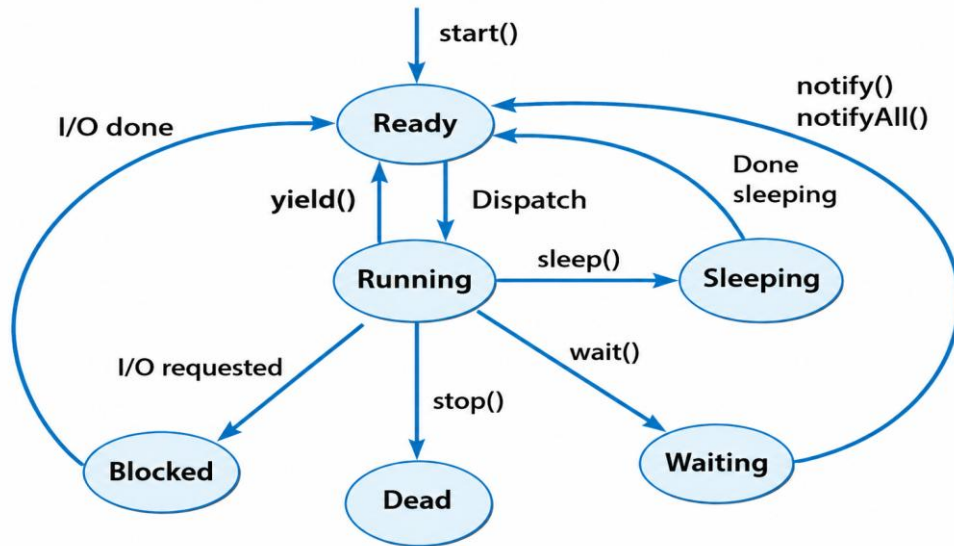
```

- **في الحل السابق كان المُنتج والمُستهلك يعتمدان على الانتظار الفعّال (Busy Waiting)**، حيث يستمر كل طرف في فحص قيمة `isAvailable` بشكل متكرر لمعرفة ما إذا كان يستطيع التنفيذ أم لا. ولتجنب هذا الهدر في وقت المُعالج تم استخدام الدالتين `wait()` و `notifyAll()`.
- **فعندما يجد المُنتج أن هناك مُنتجاً جاهزاً لم يُستهلك بعد** (`isAvailable = true`) فإنه لا يستمر بالدوران داخل المُعالج، وإنما يستدعي `wait()` لينتقل إلى حالة الانتظار (Waiting State) ويُزال من التنفيذ مؤقتاً. وبالمثل، إذا حاول المُستهلك الاستهلاك ولم يجد مُنتجاً جاهزاً فإنه يدخل أيضاً في حالة الانتظار.
- **وعند انتهاء المُستهلك من استهلاك المُنتج فإنه يجعل قيمة** `isAvailable` تساوي `false` ثم يستدعي `notifyAll()`، وبذلك يتم إرسال إشعار إلى جميع الـ Threads المُنتظرة على هذا الكائن. تنتقل هذه الـ

Threads من حالة الانتظار (Waiting) إلى حالة الجاهزية (Ready)، ثم يختار المُجدول (Scheduler) أي Thread سيحصل على المعالج أولاً.

- **ولهذا السبب استُخدمت الحلقة while بدلاً من if** ، فبعد استيقاظ الـ Thread نتيجة وصول الإشعار لا يوجد ضمان بأنه أصبح قادراً فعلاً على المتابعة، لأن Thread آخر **قد يكون استيقظ** قبله وغيّر حالة المورد. لذلك يجب إعادة التحقق من الشرط مرة أخرى بعد الاستيقاظ.
- **أما بالنسبة لاحتمال استيقاظ مُنتجين (Producers) معاً ورؤية isAvailable = false في اللحظة نفسها وإنتاج المُنتج مرتين**، فهذا لا يحدث في هذا المثال لأن الدوال ما زالت synchronized. فعلى الرغم من أن notifyAll() قد توقف عدة Threads، إلا أن القفل (Lock) لا يزال يسمح بدخول Thread واحد فقط إلى القسم الحرج (Critical Section)، بينما تبقى بقية الـ Threads مُنتظرة حتى يتم تحرير القفل وإعادة التحقق من الشرط.

## مخطط دورة حياة الخيط (Thread Life Cycle) :

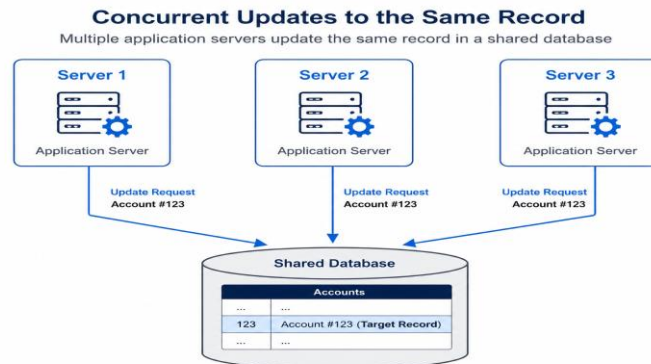


- **عند استدعاء start() ينتقل الـ Thread إلى حالة Ready**، **ويبقى** مُنتظراً دوره للتنفيذ، بعد ذلك يختار نظام التشغيل أحد الـ Threads الجاهزة ويخصص له المُعالج لينتقل إلى حالة Running، وفي حال وجود عدة أنوية، يمكن تشغيل عدة Threads بالتوازي، بينما تتناوب الـ Threads الأخرى على استخدام المُعالج عندما يُصبح مُتاحاً.
- **أثناء التنفيذ قد يعود الـ Thread إلى حالة Ready ليمنح فرصة التنفيذ لغيره**، **سواء** بسبب انتهاء الشريحة الزمنية (Time Quantum/Slice) المخصصة له أو بسبب قيامه بالتخلي عن المُعالج.
- **كما يمكن للـ Thread** أثناء التنفيذ أن يَطْلُب عمليات خاصة :
  - مثل sleep() أو wait()
  - أو أن يتوقّف عن التنفيذ نهائياً باستخدام stop()

- أو عمليات الإدخال والإخراج (I/O) كالقراءة والكتابة والطباعة.
- وتنفذ عمليات الإدخال والإخراج عادةً بواسطة مكونات أو مُحكِّمات خاصة خارج المُعالج الرئيسي، مثل مُحكِّم القرص الصلب (Disk Controller) أو مُحكِّم الشبكة (Network Controller) أو مُحكِّم لوحة المفاتيح والطابعة، لذلك عندما يُطلَب الـ Thread عملية من هذا النوع فإنه لا يبقى مشغولاً على الـ CPU،
- بل ينتقل إلى حالة Blocked بانتظار انتهاء العملية المطلوبة. وعندما تنتهي عملية الإدخال أو الإخراج يعود مرة أخرى إلى حالة Ready ليُصبح مؤهلاً للتنفيذ من جديد.
- أما عند استدعاء wait() فإن الـ Thread ينتقل إلى حالة Waiting ويبقى فيها حتى يتلقى إشعاراً بواسطة notify() أو notifyAll()، وعندها يعود إلى حالة Ready.
- وفي حالة sleep() ينتقل إلى حالة Sleeping ثم يعود إلى حالة Ready بعد انتهاء المُدة الزمنية المحددة (Timeout). أما إذا تم إيقاف الـ Thread نهائياً، سواء بانتهاء تنفيذ الدالة run() أو باستدعاء stop() (وهي دالة قديمة Deprecated وغير مُوصى باستخدامها)، فإنه ينتقل إلى حالة Terminated أو Dead، وعندها تنتهي دورة حياته ولا يمكن إعادته إلى التنفيذ مرة أخرى أو استدعاء start() عليه من جديد.
- ومن المهم ملاحظة أن عودة الـ Thread إلى حالة Ready لا تعني أن الشرط الذي كان ينتظره ما زال صحيحاً، فقد تكون حالة النظام أو البيانات قد تغيرت أثناء فترة الانتظار. لذلك يُعاد التحقق من الشرط بعد الاستيقاظ، ولهذا السبب يُستخدم غالباً while مع wait() بدلاً من if.

### ■ معالجة التنافس في حال وجود أكثر من Instance للغرض في الذاكرة :

- كانت الحلول السابقة مُناسبة عندما يكون لدينا كائن (Instance) واحد مُشترك داخل الذاكرة، حيث تمنع synchronized أكثر من Thread من تنفيذ الدوال الحساسة على الكائن نفسه في الوقت ذاته. أي إن الـ Threads ضمن التطبيق الواحد لا تستطيع الدخول معاً إلى القسم الحرج (Critical Section)، وبالتالي تتم حماية المَورد المُشترك من التنافس.
- لكن في الأنظمة المؤسسية الحقيقية كالأنظمة البنكية لا تكون المُشكلة دائماً ضمن Instance واحد داخل الذاكرة، بل قد توجد عدّة Instances للتطبيق نفسه أو عدّة خوادم تعمل مع قاعدة البيانات ذاتها كما في الصورة :

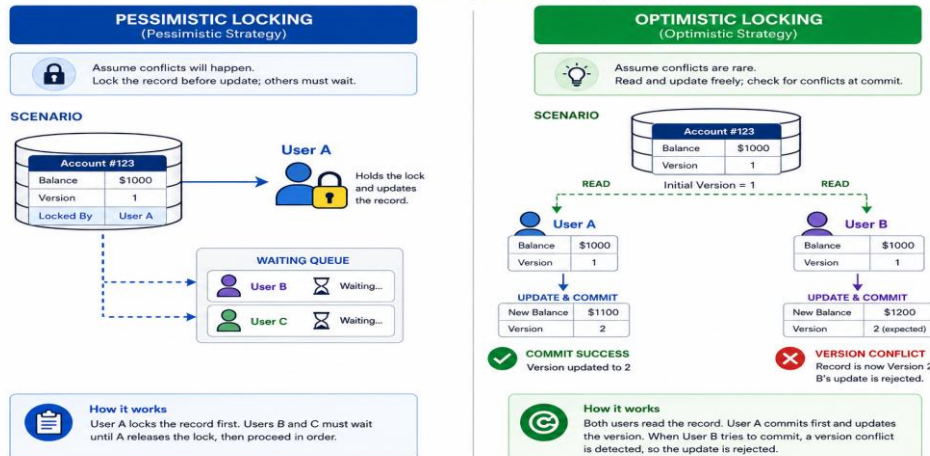


- وهنا تصبح **synchronized غير كافية**، لأن كل Server يمتلك ذاكرته الخاصة، وبالتالي لا يمكن لقفل موجود داخل عملية (Process) واحدة **أن يمنع** العمليات الموجودة على خوادم أخرى من الوصول إلى البيانات نفسها.
- في هذه الحالة تنتقل مشكلة التنافس **إلى مستوى** قاعدة البيانات، ويصبح التعامل معها قراراً تصميمياً حساساً لا يتخذ بشكل عشوائي، لأن الهدف الأساسي في تطبيقات الويب **هو الاستفادة القصوى** من موارد الحاسب والعمل بالتوازي لخدمة أكبر عدد ممكن من المستخدمين.
- لذلك **لا يكفي** أن نلاحظ أن **النظام بطيء** لنقرر مباشرة تغيير الخادم أو زيادة العتاد، بل يجب تحليل طريقة استهلاك الموارد أولاً. فإذا كان استهلاك المعالج منخفضاً رغم بطء التطبيق، فهذا يدل غالباً على وجود مشكلة في تصميم **البرنامج نفسه** أو في طريقة إدارة التنافس. وحتى إذا كان استهلاك المعالج مرتفعاً جداً فقد يكون السبب وجود انتظار فعال (Busy Waiting) أو حلقات لا نهائية تؤدي إلى هدر الموارد.
- كما أن **من المفهم** إدراك أن موارد الخوادم **ليست غير محدودة**، لذلك يجب على المهندس تقدير عدد الطلبات (Requests) التي يستطيع النظام معالجتها ضمن الموارد المتاحة. فالبرنامج الذي يعمل بشكل صحيح أثناء العرض التجريبي (Demo) ليس بالضرورة أن يكون **نظاماً صالحاً** للعمل الإنتاجي (Production). والفرق الحقيقي بينهما **لا يكمن** في الوظائف الأساسية فقط، وإنما **في القدرة** على تحقيق المتطلبات غير الوظيفية (Non-Functional Requirements) مثل التوسع (Scalability) والاستقرار (Stability) والاعتمادية (Reliability) والأداء تحت الأحمال المرتفعة. **لذلك نقاس** جودة الأنظمة المؤسسية بقدرتها على تحقيق هذه المتطلبات والمحافظة عليها مع ازدياد عدد المستخدمين والطلبات.
- كما أن تقسيم العمل **على عدة Threads** بشكل فعال **ليس أمراً بسيطاً**، لأن التوازي نفسه يضيف تكلفة ناتجة عن التواصل والتنسيق بين الـ Threads والخوادم. لذلك لابد من تحقيق توازن **بين زيادة التوازي** وبين كلفة إدارة هذا التوازي.

## استراتيجيات معالجة التنافس على مستوى قاعدة البيانات

### PESSIMISTIC vs OPTIMISTIC LOCKING

Two Concurrency Control Strategies



لِمعالجة التنافس على مُستوى قاعدة البيانات توجد استراتيجيتان أساسيتان:

#### ■ الاستراتيجية المُتَشائمَة (Pessimistic Strategy):

- **تعتمد الاستراتيجية المُتَشائمَة** على افتراض أن احتمال حدوث التنافس على المَوارِد مُرتفع، لذلك يتم منع التعارضات مُسبقاً من خلال فرض أقفال (Locks) **على البيانات أثناء** القراءة أو التعديل. فعندما يبدأ أحد المُستخدمين أو المُناقلات (Transactions) بالعمل على مَورد مُعيّن، **يُمنع الآخرون** من تعديله حتى انتهاء العملية وتحرير القفل. ويُستخدم لهذا الغرض آليات مثل (SELECT FOR UPDATE) **وغيرها** من تقنيات القفل. تُساهم هذه الاستراتيجية في منع التعارضات والحفاظ على سلامة البيانات، إلا أن الإكثار من **الأقفال** يؤدي إلى زيادة زمن الانتظار وتقليل درجة التوازي، كما يَستهلك موارد إضافية من قاعدة البيانات، مما قد يُسبب انخفاضاً في الأداء عند وجود عدد كبير من المُستخدمين **أو المُعاملات المُتزامنة**. لذلك لا يُنصح بالاعتماد على الأقفال لفترات طويلة أو على نطاق واسع.
- **مثال:** في نظام حجز تذاكر، إذا بدأ أحد المُستخدمين بحجز مقعد مُعيّن، يُمكن لقاعدة البيانات وضع قفل على سجل هذا المقعد، بحيث **لا يتمكّن أي** مُستخدم آخر من تعديله أو حجزه حتى تنتهي عملية الحجز الأولى وتثبت نتائجها.

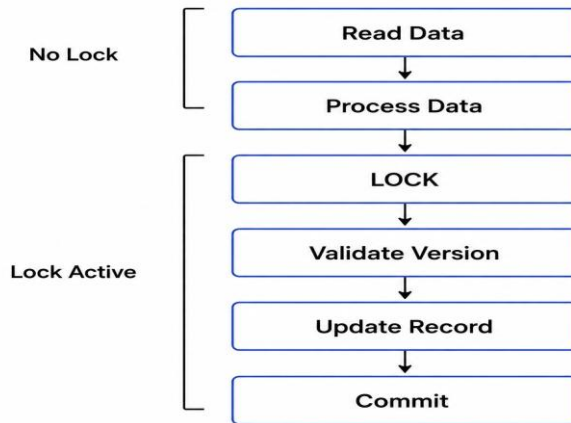
#### ■ الاستراتيجية المُتفائلة (Optimistic Strategy):

- تعتمد الاستراتيجية المُتفائلة **على افتراض أن حدوث التنافس** على المَوارِد نادر نسبياً، لذلك تسمح للمُستخدمين أو الـ Threads بالقراءة والعمل على البيانات **دون استخدام أقفال مُستمرة**. وبدلاً من ذلك يتم إرفاق رقم إصدار (Version Number) أو قيمة تحقق لكل سجل، يتم تحديثها عند كل عملية تعديل.
- عند قراءة البيانات **يتم حفظ** رقم الإصدار الحالي، وعند تنفيذ التعديل النهائي (Commit) يتحقق النظام من أن رقم الإصدار لم يتغير. فإذا كان الرقم ما يزال كما هو، فهذا يعني أن البيانات لم تُعدّل من قِبل طرف آخر، فنتم عملية الحفظ بنجاح. **أما إذا تغيّر رقم الإصدار**، فهذا يدل على أن مُستخدمًا أو مُناقلةً أخرى قامت بتعديل البيانات خلال هذه الفترة، وعندها تُرفض العملية المُتأخرة **أو يتم التراجع** عنها وإعادة تنفيذها من جديد.
- تتميز هذه الاستراتيجية بأنها تقلل الحاجة إلى الأقفال **وتسمح** بدرجة أعلى من التوازي، مما يجعلها مناسبة للأنظمة كبيرة الحجم التي تضم عدداً كبيراً من المُستخدمين، **حيث يكون عدد** عمليات القراءة أكبر بكثير من عدد عمليات التعديل.
- **مثال:** في متجر إلكتروني يحتوي على قطعة واحدة مُتبقية من مُنتج مُعيّن، يُمكن لعدة مُستخدمين قراءة بيانات المُنتج في الوقت نفسه. **وعند مُحاولَة** الشراء يتم التحقق من رقم الإصدار أو حالة المُنتج. فينجز أول مُستخدم يتم العملية، بينما تُرفض عمليات المُستخدمين الآخرين **لأن البيانات** تغيرت منذ لحظة قراءتها.
- **لا يُفضّل** استخدام الاستراتيجية المُتَشائمَة (Pessimistic Strategy) في الأنظمة كبيرة الحجم، لأنها تعتمد على الأقفال (Locks)، والأقفال **التي تستطيع** قاعدة البيانات تطبيقها على كل جدول عددها محدود. كما أن كثرة الأقفال تؤدي إلى زيادة الانتظار وتقليل درجة التوازي **بين المُستخدمين**. لذلك **يُحاول المُحلّل** تصميم النظام بحيث تكون حالات التنافس نادرة قدر الإمكان، وعندها يُمكن الاعتماد على الاستراتيجية المُتفائلة

(Optimistic Strategy) التي تسمح لعدد كبير من المُستخدمين بالعمل بالتوازي دون حجز البيانات لفترات طويلة.

## ❖ مواضيع أخرى في إدارة التنافس على مستوى قاعدة البيانات :

- كيف تعمل المُناقشات في قاعدة البيانات؟ متى تبدأ المُناقشة ومتى تنتهي ومتى يحدث القفل وما هي الأغراض التي تُقفل ومتى يتم تحريرها؟



- بعد فهم الاستراتيجيتين المُتفائلة والمُتَشائمة، يُصبح من المُهم فهم كيفية عمل المُناقشات (Transactions) داخل قاعدة البيانات. فعلى الرغم من أن المُبرمج يُحدد بداية المُناقشة ونهايتها باستخدام أوامر مثل Begin Transaction و Commit،
- إلا أن التأثير الفعلي للمُناقشة داخل قاعدة البيانات لا يبدأ منذ لحظة القراءة، وإنما يبدأ عند أول عملية تعديل (Update) على البيانات.
- تُستخدم الفهارس (Indexes) لتسريع الوصول إلى البيانات من خلال توفير طريقة أسرع للبحث داخل الجداول دون الحاجة إلى فحص جميع السجلات. إلا أن الفهرس يُعد جزءاً إضافياً من البيانات يجب المحافظة عليه وتحديثه باستمرار.
- فعندما يتم تعديل قيمة حقل مشمول بفهرس، لا تقتصر العملية على تحديث السجل فقط، بل يجب أيضاً تحديث الفهرس ليعكس القيمة الجديدة. لذلك قد تحتاج قاعدة البيانات إلى فرض أقفال ليس فقط على السجل المعدل، وإنما أيضاً على أجزاء من الفهرس المرتبط به.
- ولهذا يجب عدم إنشاء الفهارس بشكل عشوائي، لأن فائدتها تعتمد على طبيعة البيانات الموجودة في العمود. فكلما ازداد عدد القيم المُختلفة في العمود ازدادت فعالية الفهرس في تسريع البحث. أما إذا كان العمود يحتوي عدداً قليلاً من القيم المُختلفة، مثل عمود الحالة الذي يأخذ القيمتين (True/False)، فإن الفهرس لن يُساهم كثيراً في تقليل عدد السجلات التي يجب فحصها.
- وفي مثل هذه الحالات قد تتحول الفهارس من وسيلة لتحسين الأداء إلى مصدر إضافي للتنافس، لأن عدداً كبيراً من العمليات سيتعامل مع القيم نفسها داخل الفهرس. ومع زيادة

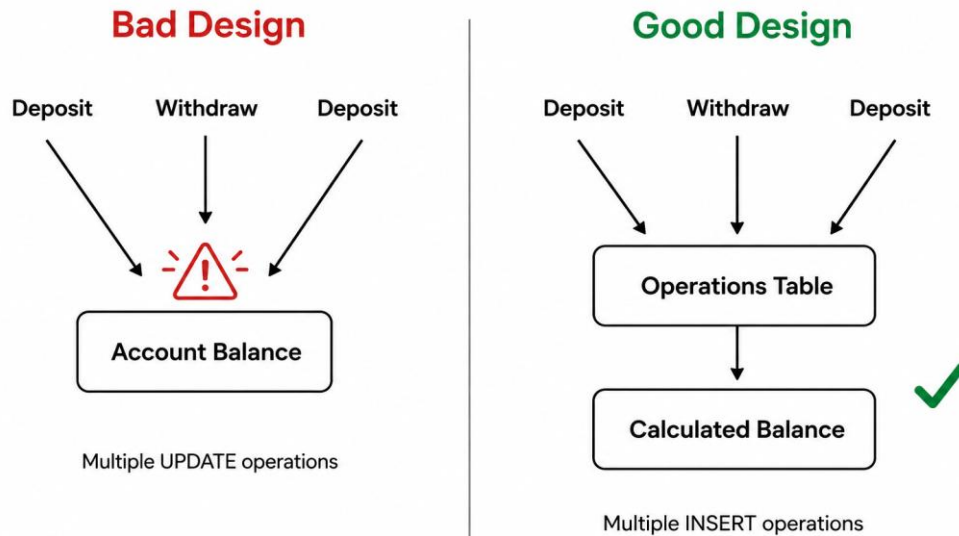
التنافس والأقفال قد تَضطر قاعدة البيانات إلى توسيع نطاق القفل، مما يؤدي إلى انخفاض التوازي وارتفاع زمن انتظار العمليات الأخرى.

■ في الاستراتيجية المُتفائلة (Optimistic Strategy) يتم تنفيذ مُعظم العمل خلال مَرحلة القراءة والمُعالجة دون حجز البيانات، ثم يُؤجَّل التعديل الفعلي إلى آخر لحظة ممكنة. وعندها فقط تبدأ قاعدة البيانات بفرض الأقفال اللازمة لإجراء التحديث وكتابة النتائج ثم إنهاء المُناقلة (Commit). وبهذه الطريقة يتم تقليل مِدَّة المُناقلة وتقليل زمن بقاء الأقفال، مما يسمح بدعم عدد أكبر من المُستخدمين والعمل بدرجة أعلى من التوازي.

### تذكرة

- تُؤدي زيادة عدد الفهارس (Indexes) إلى تحسين وتسريع عمليات القراءة (Read)، لأن قاعدة البيانات تستطيع الوصول إلى البيانات المطلوبة بسرعة أكبر دون الحاجة إلى فحص عدد كبير من السجلات. لكن في المقابل تُصبح عمليات الكتابة (Insert, Update, Delete) أبطأ، لأن قاعدة البيانات لا تُعدّل البيانات الأصلية فقط، وإنما تَضطر أيضاً إلى تحديث جميع الفهارس المُرتبطة بها.
- فكل عملية تعديل قد تتطلب تحديث السجل نفسه بالإضافة إلى تحديث عدّة فهارس، مما يعني تنفيذ عمليات كتابة إضافية وفرض أقفال (Locks) على البيانات والفهارس المُرتبطة بها.

### مثال واقعي على تقليل التنافس



- من الأمثلة المُهمّة على تقليل التنافس في قواعد البيانات تصميم الحسابات المصرفية. قد يظن المُبرمج المبتدئ أن الحل الأبسط هو وجود عمود balance في جدول الحساب، بحيث يتم تعديل هذا الرصيد مع كل عملية سحب أو إيداع. لكن هذا التصميم يُسبب مُشكلة، لأن كل العمليات المُتزامنة ستتنافس على تعديل السجل نفسه، أي أن كل عملية سَتُنفَّذ Update على الحساب ذاته، مما يزيد احتمال حدوث التنافس ويُقلّل الأداء.

- لذلك يكون التصميم الأفضل هو تسجيل العمليات في جدول أو حقل مُستقل للعمليات Operations، بحيث لا تُعدّل الرصيد مُباشرة مع كل عملية، بل تُضيف عملية جديدة مثل: إيداع 10 أو سحب 20. وبهذا تُسَبَدل كَثرة عمليات التعديل Update بعمليات إدخال Insert، وهي غالباً أقلّ تسبباً بالتنافس لأنها لا تُعدّل السجل نفسه في كُلّ مرة. كما أن هذا التصميم يُوفّر سجلاً تاريخياً واضحاً يُبين ما الذي غيّر الرصيد، بدل أن يكون لدينا رقم نهائي فقط لا نعرف كيف وصلنا إليه.
- وعند الحاجة إلى معرفة الرصيد، يتم حسابه اعتماداً على مجموع العمليات المسجلة. و لتحسين الأداء لا نحتاج إلى إعادة حساب جميع العمليات منذ البداية في كل مرة، بل يمكن حفظ آخر رصيد محسوب وآخر نقطة تم الحساب عندها، ثم إضافة العمليات الجديدة فقط التي حدثت بعد تلك النقطة. وبهذه الطريقة نقلل عمليات التعديل على السجل المشترك، ونخفف التنافس، ونحافظ في الوقت نفسه على تاريخ العمليات وسلامة البيانات.

#### ملاحظة:

كُلّ الحلول التصميمية التي تمنع التنافس في أنظمة **المُستودعات** والمخزون يمكن تطبيقها أيضاً على الأموال والعملات، **لأن كليهما يُمثّل كمية تتغير مع الزمن نتيجة مجموعة من العمليات.**

- **من المبادئ الفهمية** في الأنظمة المؤسسية تقليل مدّة بقاء الأقفال إلى الحد الأدنى المُمكن. فعندما تبدأ عملية التعديل (Update) على سجل داخل قاعدة البيانات، **يتم فرض القفل** على هذا السجل لحمايته من التعديلات المُتزامنة. ولا يتحرر هذا القفل عند انتهاء عملية الكتابة مُباشرة، وإنما يبقى موجوداً حتى انتهاء المُناقلة (Transaction) وتنفيذ عملية الـ Commit أو الـ Rollback.
- **لذلك لا يكفي تأجيل** أول عملية تعديل إلى آخر لحظة فقط، بل يُفضّل أيضاً تجميع جميع عمليات التعديل والكتابة في نهاية الخدمة أو العملية البرمجية **قدر الإمكان**. فكلما بدأ التعديل مبكراً، **يَقِي القفل** مفروضاً لفترة أطول، مما يزيد زمن انتظار العمليات الأخرى ويُقلّل درجة التوازي داخل النظام.
- ولهذا السبب نجد أن التصميم السليم في التطبيقات المؤسسية يعتمد على **تنفيذ مراحل القراءة** والتحقق والمعالجة والحسابات أولاً، ثمّ تنفيذ جميع عمليات Insert و Update و Delete **بشكل مُتتابع** في نهاية العملية. وبهذه الطريقة تبقى الأقفال فعالة لأقصر مدة ممكنة.
- **أما بالنسبة** للبيانات التي تم تعديلها داخل المُناقلة، **فإن المُناقلة** نفسها تستطيع رؤية هذه التعديلات والاستمرار بالعمل عليها قبل تنفيذ الـ Commit. أما قدرة المُناقلات الأخرى على رؤية هذه التعديلات فتتوقف على مُستوى العزل (Isolation Level) المُستخدم في قاعدة البيانات.

**الإنسان في هذه الحياة؛ إما أن يكون عبدة لمن يُعتبر، أو أن يكون هو من يُعتبر.**

## مُستويات العزل في قاعدة البيانات (Isolation Levels)

يُحدد مُستوى العزل (Isolation Level) ما إذا كانت المُناقشات الأخرى تستطيع رؤية التعديلات التي تمت داخل مناقلة لم تنتهِ بعد. ولدينا 3 مستويات على مستوى قاعدة البيانات :

### 1. Dirty Read:

- في هذا المستوى يُسمح للمُناقلة بقراءة البيانات المُعدّلة حتى لو لم يتم تنفيذ Commit بعد. لذلك قد يقرأ أحد المُستخدمين قيمة جديدة ثم تقوم المُناقلة الأصلية بتنفيذ Rollback، فتختفي هذه القيمة وتعود البيانات إلى حالتها السابقة. ولهذا السبب يُسمى هذا النوع "قراءة مُتسخة" لأنه يسمح بقراءة بيانات غير مُؤكّدة بعد.
- ورغم أن هذا المُستوى قد يؤدي إلى قراءة قيم غير دقيقة، إلا أنه يكون مقبولاً في بعض الحالات التي لا تتطلب دقة عالية، مثل الإحصائيات أو القيم التقديرية (عدد الولادات في المنطقة...) التي لا يؤثر تغييرها البسيط على القرار النهائي.

### 2. Read Committed:

- في هذا المُستوى لا يُسمح للمُناقشات الأخرى برؤية التعديلات إلا بعد تنفيذ Commit وإذا أرادت مُناقلة القراءة فستقرأ القيمة القديمة.
- ولتحقيق ذلك تحتفظ قاعدة البيانات بنسخ مُناسبة من البيانات القديمة حتى تتمكن العمليات الأخرى من مُتابعة القراءة دون رؤية التعديلات غير المُؤكّدة.
- ولهذا السبب تزداد كلفة إدارة البيانات والذاكرة مُقارنة بمُستوى Dirty Read.
- وتعتمد قواعد البيانات على آليات مثل سِجلات التراجع (Rollback Segments) للاحتفاظ بالقيم القديمة وإمكانية استعادتها عند تنفيذ Rollback أو توفير النسخة المُناسبة للقراءة أثناء تنفيذ المُناقشات الأخرى.
- لذلك تُعدّ المُناقشات الطويلة من أسوأ الممارسات في الأنظمة المؤسسية، لأنها تُبقي الأقفال والنسخ المؤقتة لفترات طويلة، مما يزيد استهلاك الموارد ويؤدي إلى انخفاض أداء قاعدة البيانات. ولهذا يُنصح دائماً بأن تكون المُناقشات قصيرة قدر الإمكان، بغض النظر عن نوع المُهمّة التي تُنفّذها.

### 3. Serializable:

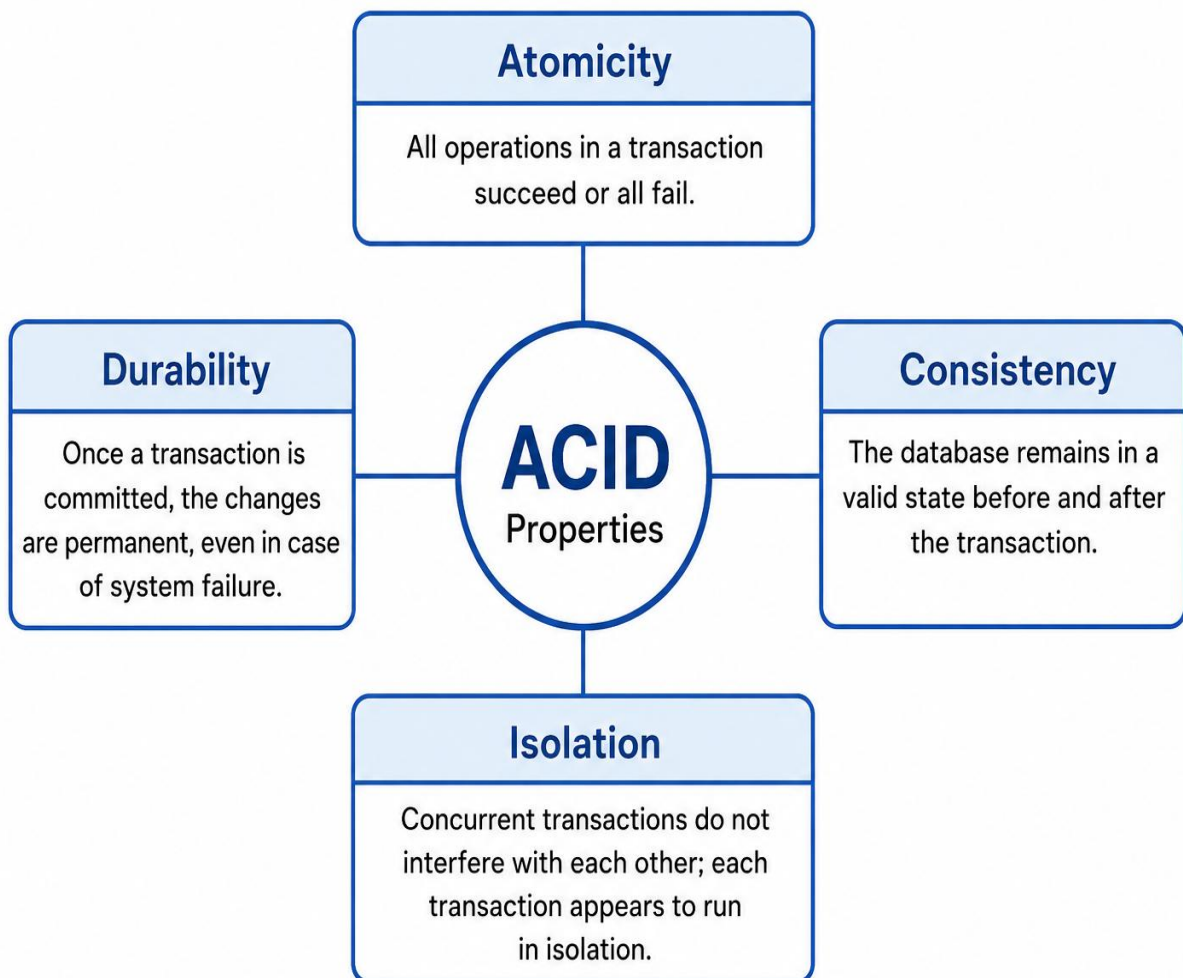
- مُستوى عزل يضمن أن نتيجة تنفيذ المُناقشات المُتزامنة تكون مكافئة تماماً لنتيجة تنفيذها بشكل تسلسلي (واحدة بعد الأخرى)
- لذلك يُعدّ أعلى مُستويات العزل وأكثرها أماناً، لكنه يأتي على حساب الأداء وتقليل درجة التوازي داخل النظام.

**مثال:** في نظام بنكي ما، لنفترض أن قيمة الرصيد (Balance) كانت 50، ثم قامت إحدى المُعاملات بتعديلها إلى 100 لكنها لم تُنفَّذ Commit بعد:

- Dirty Read: المُعاملَة الأُخرى تقرأ القيمة 100.
- Read Committed: المُعاملَة الأُخرى تقرأ القيمة 50.
- Serializable: المُعاملَة الأُخرى لا تقرأ لا 50 ولا 100 مباشرة، وإنما تنتظر انتهاء المُعاملَة الأولى (Commit أو Rollback)، ثم تُتابع عملها وكأنَّ المُعاملتين تم تنفيذهما واحدة بعد الأُخرى.

### تذكرة:

تتميز المُعاملات (Transactions) بمجموعة من الخصائص الأساسية المعروفة باسم ACID، والتي تضمن صحة البيانات وسلامة تنفيذ العمليات داخل قاعدة البيانات.



## 1. Atomicity (الذرية):

تعني أن المُناقلة **تُنَفَّذُ كَ وَحْدَةٍ وَاحِدَةٍ** غير قابلة للتجزئة، **فإما** أن تنجح جميع العمليات الموجودة داخلها ويتم تنفيذها بالكامل، **أو** تفشل جميعها ويتم التراجع عنها (Rollback). فلا يجوز أن يُنفَّذ جزء من المُناقلة بينما يفشل الجزء الآخر.

## 2. Consistency (التجانس أو الاتساق):

تعني أن قاعدة البيانات **تَبْقَى فِي حَالَةٍ صَحِيحَةٍ** ومُتجانسة **قَبْلَ** تنفيذ المُناقلة **وَبَعْدَ** انتهائها. أي أن المُناقلة لا يجوز أن تترك البيانات في حالة مُتناقضة أو مُخالفة لقواعد العمل (Business Rules). فمثلاً لا يجوز أن يؤدي تنفيذ عملية شراء بقيمة 100 إلى إنقاص رصيد العميل بمقدار 120. **كما تكتسب** هذه الخاصية أهمية خاصة في الأنظمة التي تعتمد على تسجيل العمليات ثم اشتقاق القيم النهائية منها، **حيث يُمكن** الرجوع إلى سجل العمليات وإعادة حساب النتائج والتحقق من صحتها في أي وقت.

## 3. Isolation (العزل):

تعني أن كُل مُناقلة تعمل **وكأنها الوحيدة** الموجودة في النظام، بحيث **لا تتأثر بالمُناقلات الأخرى** أثناء التنفيذ. ويتم التحكم بدرجة العزل من خلال مُستويات العزل (Isolation Levels) المُختلفة مثل Dirty Read و Read Committed وغيرها.

## 4. Durability (الديمومة):

تعني أن نتائج المُناقلة **تُصبح دائمة بعد تنفيذ عملية Commit**. فبمجرد تثبيت التعديلات لا يجوز فقدانها حتى في حال حدوث عطل أو إعادة تشغيل للنظام. **وبالمقابل**، إذا تم تنفيذ Rollback فإن جميع التعديلات داخل المُناقلة تُلغى نهائياً وتعود البيانات إلى حالتها السابقة ولا نستطيع بعدها القيام بـ Commit أبداً.

## المُناقلات الكبيرة في الأنظمة المؤسسية

- **عند تنفيذ عمليات ضخمة** مثل حذف مليون سجل أو استيراد آلاف السجلات دفعة واحدة، فإن تنفيذها ضمن مُناقلة واحدة كبيرة يؤدي إلى تضخم الـ Rollback Segment، لأن قاعدة البيانات تحتاج إلى الاحتفاظ بالمعلومات اللازمة للتراجع عن كل عملية في حال فشل التنفيذ. **وهذا يُسبب بطئاً** شديداً، ويزيد مدة بقاء الأقفال، ويمنع العمليات الأخرى من العمل بكفاءة على قاعدة البيانات.
- **لذلك يُستخدم أسلوب المُعالجة الكسولة** أو المُجزأة (Lazy / Batch Processing)، بحيث لا تُنفَّذ العملية كُلّها دفعة واحدة، بل تُقسَّم إلى دفعات صغيرة. **فبدل** من حذف مليون سجل في مُناقلة واحدة، يُمكن حذف 100 سجل ثم تنفيذ Commit، ثم حذف 100 سجل أخرى، وهكذا حتى تنتهي العملية. وبذلك يبقى حجم الـ

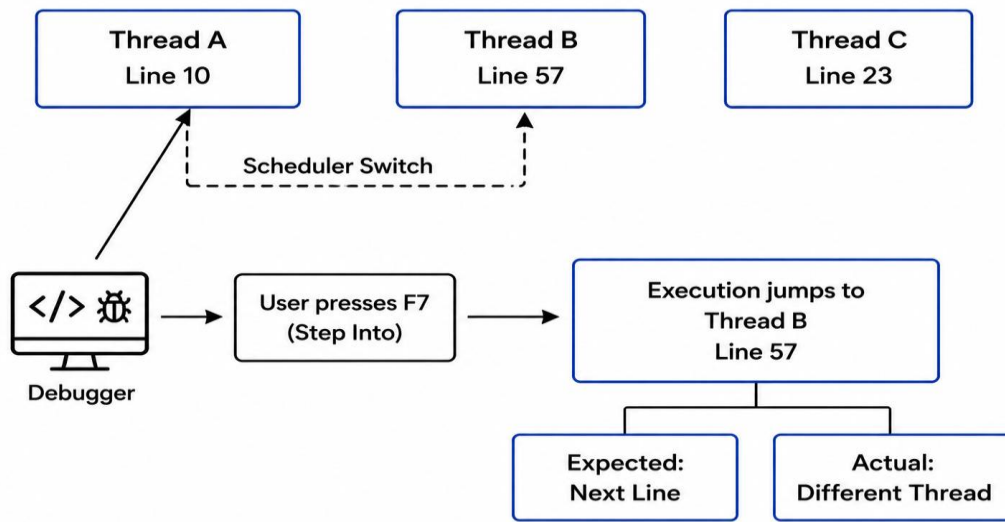
Rollback Segment صغيراً في كل دفعة، وتبقى المناقلات قصيرة، وتتاح فرصة أكبر للعمليات الأخرى كي تعمل على قاعدة البيانات.

- **وينطبق الأمر نفسه على عمليات الاستيراد الكبيرة**، فبدل استيراد 10 آلاف سجل دفعة واحدة، يتم استيرادها على دفعات، مع حفظ مؤشر أو سجل يوضح آخر نقطة وصل إليها التنفيذ، **حتى يمكن متابعة** العمل من المكان الصحيح في حال حدوث توقف أو خطأ. وبهذا نحافظ على خاصية Atomicity داخل كل دفعة صغيرة، ونُقل أثر العملية الضخمة على أداء النظام ككل.
- **تعدّ خاصية Atomicity من أهم الأسباب التي تدفع قاعدة البيانات** إلى الاحتفاظ بمعلومات التراجع (Undo/Rollback Information)، إذ يجب أن تكون قادرة على إعادة البيانات إلى حالتها السابقة عند فشل المناقلة. وكلما زاد حجم المناقلة وعدد العمليات التي تحتويها، **ازداد حجم** معلومات التراجع المطلوبة، مما يؤدي إلى زيادة استهلاك الموارد وانخفاض الأداء. لذلك يُنصح بأن تكون المناقلات قصيرة قدر الإمكان. ولا تقتصر أهمية تقليل حجم المناقلات على عمليات الحذف والاستيراد الضخمة فقط، **بل تمتد** أيضاً إلى عمليات القراءة والتقارير، إذ إن فتح مُناقلة دون حاجة حقيقية يؤدي إلى زيادة تكلفة العزل وإطالة مدة احتفاظ قاعدة البيانات بالموارد اللازمة لتنفيذها.
- **بشكل عام لا يُنصح بفتح مُناقلة (Transaction)** إلا عند الحاجة الفعلية إليها، **لأن** وجود مُناقلة فعّالة يعني أن قاعدة البيانات ستضطر إلى توفير درجة أعلى من الحماية والعزل للبيانات.

■ وعند تنفيذ عملية القراءة داخل مُناقلة قد نحصل على ما يُسمى :

- **Transactional Read**: أي قراءة مجموعة من البيانات **بشكل ثابت ومتسق** طوال فترة القراءة، بحيث لا تؤثر عليها التعديلات التي قد تجريها المناقلات الأخرى أثناء تنفيذ العملية ( أي يتم وضع قفل على الكتابة ).
- وتظهر أهمية ذلك في بعض الحالات الخاصة، مثل التقارير المالية أو العمليات الحساسة التي تتطلب الحصول على صورة ثابتة للبيانات خلال كامل عملية القراءة. **فعلى سبيل المثال**، إذا كان التقرير يقرأ جميع العمليات المنفذة خلال فترة زمنية معينة، فقد يؤدي إدخال عمليات جديدة أثناء القراءة إلى نتائج غير متجانسة. ومع ذلك فإن هذا النوع من القراءة مُكلف من ناحية الأداء ويستخدم بحذر، **لذلك** فإن معظم التقارير لا تعتمد عليه إلا عند الضرورة.
- **وفي الأنظمة المؤسسية الكبيرة** تُعالج بعض الحالات الحرجة بطريقة مختلفة. فعند تنفيذ مهام مثل إغلاق السنة المالية، لا يُعتمد فقط على آليات العزل داخل قاعدة البيانات، وإنما يتم منع المُستخدمين من إجراء عمليات كتابة على مستوى النظام بالكامل حتى انتهاء المهمة، وذلك لتجنب التأثير على البيانات أثناء تنفيذ العمليات الحساسة.
- **كما يُنصح بتجنب المناقلات الطويلة** وتقسيم الأعمال الضخمة **إلى مجموعة من المناقلات الصغيرة المُستقلة**. فبدل تنفيذ عملية تستغرق ساعات أو تتعامل مع ملايين السجلات ضمن مُناقلة واحدة، يتم تقسيمها إلى دفعات صغيرة مع **حفظ** نقطة التقدّم **التي وصل إليها التنفيذ**. وبهذه الطريقة، إذا حدث انقطاع للكهرباء أو نفاذ للذاكرة أو أي خطأ غير مُتوقع، يُمكن إصلاح المشكلة ومتابعة العمل من آخر نقطة ناجحة، **بدل إعادة** تنفيذ العملية كاملة من البداية. وهذا يُقلل زمن الاستعادة ويحسن موثوقية النظام وأداء قاعدة البيانات.

## تحديات تصحيح البرامج المتوازية (Parallel Program Debugging)



- **يُعدّ تنقيح وتصحيح البرامج المتوازية (Debugging)** من أصعب المهام البرمجية، لأنّ سلوك البرنامج لا يعود قابلاً للتنبؤ بشكل كامل كما كان الحال في البرامج أحادية الخيط (Single Threaded). ففي البرنامج العادي، إذا كانت قيمة متغير  $x=10$  في سطر مُعيّن، فمن المُتوقّع أن تُصبح 11 بعد تنفيذ التعليمة  $x = x + 1$ .
- **لكن في البيئات مُتعددة الخيوط**، فقد يتمكّن Thread آخر من تعديل القيمة **نفسها بين هاتين** التعليمتين، وبالتالي لا يعود المُبرمج قادراً على افتراض أنّ قيمة المُتغيّر بقيت ثابتة أثناء التنفيذ.
- **كما أنّ التنفيذ لا ينتقل دائماً بشكل خطي** من سطر إلى السطر الذي يليه. فقد ينتقل المُعالج في أي لحظة إلى Thread آخر، مما يجعل تتبع مسار التنفيذ وفهم سبب الخطأ **أمرأ أكثر تعقيداً**. وحتى عند إيقاف جميع الـ Threads الأخرى لمراقبة Thread واحد فقط، قد يتوقّف البرنامج عن العمل بصورة صحيحة لأن هذا الـ Thread قد يكون مُعتمداً على Threads أخرى لتنفيذ جزء من المهمّة.
- **لهذا السبب تُصبح أدوات الـ Debugging** التقليدية أقل فعالية في الأنظمة المتوازية، ويُعتمد غالباً على أساليب أخرى **مثل الـ Tracing والـ Logging**، حيث يتم تسجيل الأحداث المهمّة وتسلسل تنفيذ العمليات، مما يُساعد على تحليل سلوك النظام واكتشاف أسباب الأخطاء بعد حدوثها.

### ما دور الـ partitioning في التخفيف من التضارب ؟

- يُستخدم الـ Partitioning لتقليل التنافس وتحسين أداء الجداول كبيرة الحجم. ففي الأنظمة المؤسسية قد تحتوي قاعدة البيانات على مئات الجداول، لكن غالباً يوجد عدد محدود من الجداول يستحوذ على معظم عمليات الإدخال والتعديل ويزداد حجمه بسرعة كبيرة مع الزمن، لذلك يصل إلى أحجام أكبر بكثير من بقية الجداول، بينما تبقى بقية الجداول صغيرة ومرجعية مثل جداول الدول والعملات. لذلك تتركز معظم عمليات القراءة والكتابة والأقفال على هذه الجداول الضخمة.

- عند زيادة عدد الأقفال على جدول معين قد تضطر قاعدة البيانات إلى توسيع نطاق القفل، وفي بعض الحالات قد يصل الأمر إلى قفل الجدول بالكامل، مما يُقلّل عدد العمليات التي يُمكن تنفيذها بالتوازي. و لتجنب ذلك يتم تقسيم الجدول الكبير إلى عدة Partitions، وغالباً يكون التقسيم حسب التاريخ، لأنّ معظم الاستعلامات تهتم بالبيانات الحديثة مثل بيانات اليوم أو الأسبوع الأخير.
- ومن الناحية المنطقية يبقى الجدول جدولاً واحداً بالنسبة للمُبرمج، لكنه فيزيائياً يُقسّم إلى عدّة Partitions مُستقلّة، فيُصبح وكأنّه عدّة جداول بدل جدول واحد. ونتيجة لذلك لا تتركز جميع العمليات والأقفال على جزء واحد من البيانات، بل تتوزع على عدّة أجزاء. لذلك تُصبح عمليات البحث أسرع، وحتى عند الاضطرار إلى تنفيذ Full Scan فإنه غالباً يقتصر على Partition مُحدد بدل المرور على جميع البيانات.
- البيانات على الوصول مُباشرة إلى الجزء المطلوب من البيانات. وبذلك يقل نطاق الأقفال، ويصبح احتمال قفل الجدول بالكامل أقل، مما يزيد درجة التوازي ويُقلل احتمالية حدوث الاختناقات أو حالات ال Deadlock في الجداول الضخمة.

**يتبيّن من هذه المحاضرة أنّ إدارة التنافس لا تعتمد على أداة واحدة، وإنما على مجموعة من التقنيات المتكاملة تبدأ من تنظيم عمل ال Threads داخل التطبيق، مروراً باستراتيجيات التحكم بالتنافس على مُستوى قاعدة البيانات، وانتهاءً بالمناقلات ومُستويات العزل وتقنيات تحسين الأداء، مثل Partitioning وغيرها. ويهدف ذلك كله إلى تحقيق التوازن بين سلامة البيانات و الأداء، وقابلية التوسّع.**



انتهت المحاضرة بالتوفيق...